Aaron N. Cutshall, DHA, MSHI

Common Table Expressions

SQLRV, LLC
HEALTHCARE INFORMATICS
https://www.sqlrv.com

Just who is this guy?

SQLRV, LLC
HEALTHCARE INFORMATICS

Principal Data Architect, Population Health

BYU
PATHWAY
WORLDWIDE

Adjunct Instructor

redgate AMBASSADOR
SYNCARI ADVISOR
torc

Ambassador Programs

DGIQ + EDW
DATA SUMMIT
UNLEASH THE POWER OF YOUR DATA
DATATUNE

Speaker – various events

B.S. Computer Science

M.S. Computer Information Systems

M.S. Health Informatics

VIRGINIA UNIVERSITY OF LYNCHBURG
SIBI AUXILIUM · ET LIBERTAS
1886

Doctor of Healthcare Administration

40 Years

Copyright © 2026, SQLRV LLC - All Rights Reserved

Something to consider…

"Common sense is seeing things as they are; and doing things as they ought to be."

– Harriet Beecher Stowe 1811-1896 (author of *Uncle Tom's Cabin*)

Copyright © 2026, SQLRV, LLC - All Rights Reserved

Agenda

- What are Common Table Expressions?
  - Purposes and Uses
  - How They Are Created
- How to Use
  - Single and Multiple CTEs
  - Recursive CTE
- Best Practices
  - Tips & Tricks
  - Things to Keep in Mind
- Closing Thoughts

Copyright © 2026, SQLRV, LLC - All Rights Reserved

Definition of a Common Table Expression

- Introduced to SQL Server 2005
- Temporary result set (not stored)
  - Within the scope of a single statement
  - *Expression* that acts like a derived *table*
  - *Common* (applicable) to the entire query
- Can be used instead of sub-queries
- Used to improve readability and clarity

Copyright © 2026, SQLRV, LLC - All Rights Reserved

Syntax of a CTE

```
-- Syntax for SQL Server, Azure SQL

[ WITH <common_table_expression> [ ,...n ] ]

<common_table_expression>::=
    expression_name [ ( column_name [ ,...n ] ) ] AS
    ( CTE_query_definition )
```

Syntax of a CTE

- *expression_name* – Name for CTE
  - Must be unique within entire query
  - Any reference uses the CTE and not the base object
- *column_name* – Name for each CTE column
  - No duplicate names
  - # of columns must match # of columns in expression
  - List is optional if all columns in expression are named
- *CTE_query_definition* – SELECT statement for CTE
  - Must meet same requirements as for creating a view
  - Multiple queries must be separated with set operators

Copyright © 2026, SQLRV, LLC - All Rights Reserved

Example of a CTE

```sql
-- Define the CTE expression name and column list
WITH cteSales (SalesPersonID, SalesOrderID, SalesYear) AS (
    -- Define the CTE query
    SELECT SalesPersonID, SalesOrderID, YEAR(OrderDate)
    FROM Sales.SalesOrderHeader
    WHERE SalesPersonID IS NOT NULL
    )
-- Define the outer query referencing the CTE name
SELECT SalesPersonID, COUNT(SalesOrderID) AS TotalSales, SalesYear
FROM cteSales
GROUP BY SalesYear, SalesPersonID
ORDER BY SalesPersonID, SalesYear;
```

Guidelines for Creating and Using CTEs

- A CTE must be followed by a single statement:
  - SELECT, INSERT, UPDATE, or DELETE
  - References some or all the CTE columns
  - Can also be specified in a CREATE VIEW statement
- Multiple CTE query definitions are allowed with a set operator: UNION ALL, UNION, INTERSECT, or EXCEPT

Copyright © 2026, SQLRV, LLC - All Rights Reserved

Guidelines for Creating and Using CTEs

- A CTE can reference:
  - Itself (in a recursive CTE)
  - Previously defined CTEs in the same WITH clause
  - Forward referencing is not allowed
- Only a single WITH statement is allowed
- A CTE cannot use the following:
  - ORDER BY (except when a TOP clause is specified)
  - INTO
  - OPTION clause with query hints
  - FOR BROWSE

Copyright © 2026, SQLRV, LLC - All Rights Reserved

Guidelines for Creating and Using CTEs

- A statement previous to a CTE must be followed by a semicolon
- A query referencing a CTE can be used to define a cursor
- Remote server tables can be referenced in the CTE
- An error occurs when query hints referencing a CTE conflict with hints within the CTE

Copyright © 2026, SQLRV, LLC - All Rights Reserved

Demo 1

Copyright © 2026, SQLRV, LLC - All Rights Reserved

Recursive CTE

- *Recursion* is where a process executes itself
- Functions make recursive calls backwards
  - Starts with a known value
  - Works back to a base case
- Factorial sequence:
  - Base case: $F_1 = 1$
  - $F_n = n * F_{n-1}$
  - See Steve Stedman's post

Copyright © 2026, SQLRV, LLC - All Rights Reserved

Recursive CTE

- A rCTE can in effect “call” itself
  - A base query joined with “recursive” query
  - Works forward, not in reverse as functions
    - Starts with a base case
    - Works forward through completion of data
- Used most often in hierarchical situations
  - Employees and Managers
  - Bill of Materials
  - Kits and Assemblies

Copyright © 2026, SQLRV, LLC - All Rights Reserved

Recursive CTE (example)

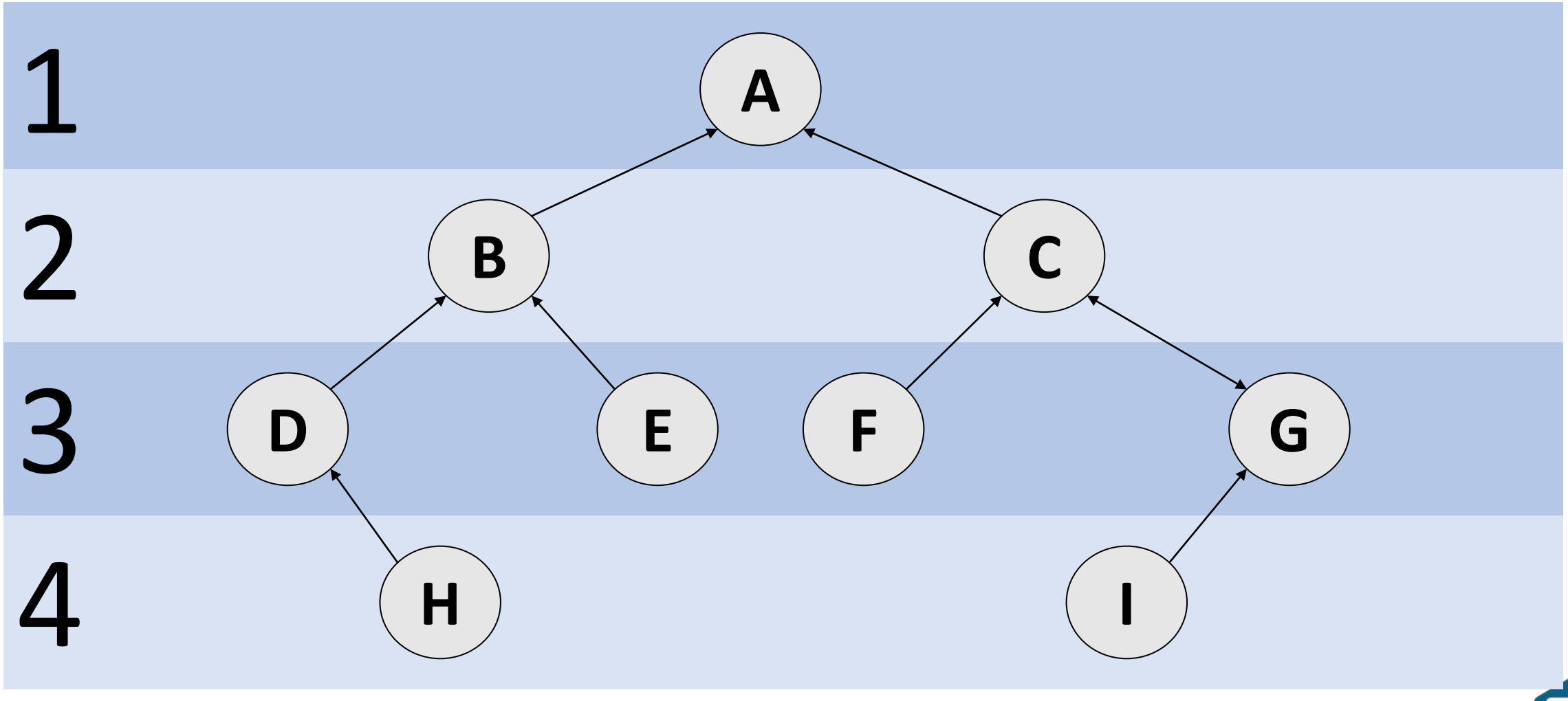
```sql
-- Declare the recursive CTE
WITH cteManagers(ManagerID, EmployeeID, Title, EmployeeLevel) AS (
    -- Declare the base case
    SELECT ManagerID, EmployeeID, Title, 0 AS EmployeeLevel
    FROM dbo.MyEmployees
    WHERE ManagerID IS NULL
    -- Combine both parts with UNION ALL
    UNION ALL
    -- Declare the recursive case
    SELECT e.ManagerID, e.EmployeeID, e.Title, m.EmployeeLevel + 1
    FROM dbo.MyEmployees AS e
    INNER JOIN cteManagers AS m
        ON m.EmployeeID = e.ManagerID
    )
SELECT ManagerID, EmployeeID, Title, EmployeeLevel
FROM cteManagers
ORDER BY ManagerID;
```

Recursive CTE

- A rCTE is essentially a loop to UNION the following:
  - CEO who has no manager (base case)
  - Employees that report to the CEO (first level)
  - Employees that report to the 1st level managers
  - Employees that report to the 2nd level managers
  - etc.
- ***<u>** NOT SET BASED **</u>***

Copyright © 2026, SQLRV, LLC - All Rights Reserved

Recursive CTE



Copyright © 2026, SQLRV, LLC - All Rights Reserved

Guidelines for Recursive CTEs

- A rCTE must contain at least two query definitions:
  - One or more base (anchor) members
  - One or more recursive members
- Multiple base and recursive members can be defined:
  - All base members are declared first
  - Recursive members are declared second
  - Only a recursive member references the rCTE

Copyright © 2026, SQLRV, LLC - All Rights Reserved

Guidelines for Recursive CTEs

- Base members must be combined by:
  - UNION (ALL)
  - INTERSECT
  - EXCEPT
- Only UNION ALL is allowed between last base member and first recursive member
- Number and data type of columns in all members must match
- A recursive member can only reference CTE once

Copyright © 2026, SQLRV, LLC - All Rights Reserved

Guidelines for Recursive CTEs

- Not allowed in query of a recursive member:
  - SELECT DISTINCT or TOP
  - GROUP BY or HAVING
  - PIVOT
  - Scalar aggregation
  - LEFT, RIGHT or FULL OUTER JOIN (INNER JOIN is OK)
  - Subqueries
  - Query hints on recursive reference

Copyright © 2026, SQLRV, LLC - All Rights Reserved

Guidelines for Recursive CTEs

- All returned columns are considered nullable
- Incorrect rCTEs may cause an infinite loop
- Limit recursion levels with MAXRECURSION hint:
  - Valid values between 0 and 32,767 (integer)
  - System default is 100
  - When 0 is specified, no limit is applied
  - Only one hint is allowed

Copyright © 2026, SQLRV, LLC - All Rights Reserved

Guidelines for Recursive CTEs

- A view with a rCTE cannot modify data
- Cursors may be defined on queries using rCTEs
  - Used only for a SELECT statement
  - Only fast forward-only and static (snapshot) allowed
  - Any other cursor type is converted to static
- Analytic and aggregate functions in recursive part
  - Apply only to current recursion level
  - Not to the whole set

Copyright © 2026, SQLRV, LLC - All Rights Reserved

Demo 2

Copyright © 2026, SQLRV, LLC - All Rights Reserved

Tips & Tricks

- Create a tally table on the fly
- Delete duplicate records
- Break strings into multiple segments
- Identify missing data
- Fibonacci sequence:
  - Base case: $F_0 = 0$; $F_1 = 1$
  - $F_n = F_{n-1} + F_{n-2}$
  - Code Project Example

Copyright © 2026, SQLRV, LLC - All Rights Reserved

Demo 3

Copyright © 2026, SQLRV, LLC - All Rights Reserved

Things to Keep in Mind

- CTE and the semi-colon:
  - Previous statement must be terminated
  - It is NOT the same as a prefix!!
  - Use semi-colons regularly for all statements
- Improve Clarity:
  - Use CTE to reduce confusion, not create it
  - Use appropriate naming conventions
  - Be consistent in usage

Copyright © 2026, SQLRV, LLC - All Rights Reserved

Things to Keep in Mind

- Benefits over derived tables/inline views:
  - Top Down Programming
    - Declare query before use
    - Can execute CTEs individually
  - Improved Readability
    - Separate query from a big mess into readable chunks
    - Provide more room for inline comments
  - Check out SQLServerCentral.com for more

Copyright © 2026, SQLRV, LLC - All Rights Reserved

Things to Keep in Mind

- Performance considerations:
  - Avoid querying data unnecessarily
    - Filter records before additional processing
    - Identify columns needed to avoid unnecessary I/O
  - Avoid multiple references where possible
    - Code consolidation makes reuse of query easier when appropriate
    - Causes re-execution of query for each reference
    - Complex queries should use temp tables or table variables
  - Recursive CTE is a loop: Not Set Based

Copyright © 2026, SQLRV, LLC - All Rights Reserved

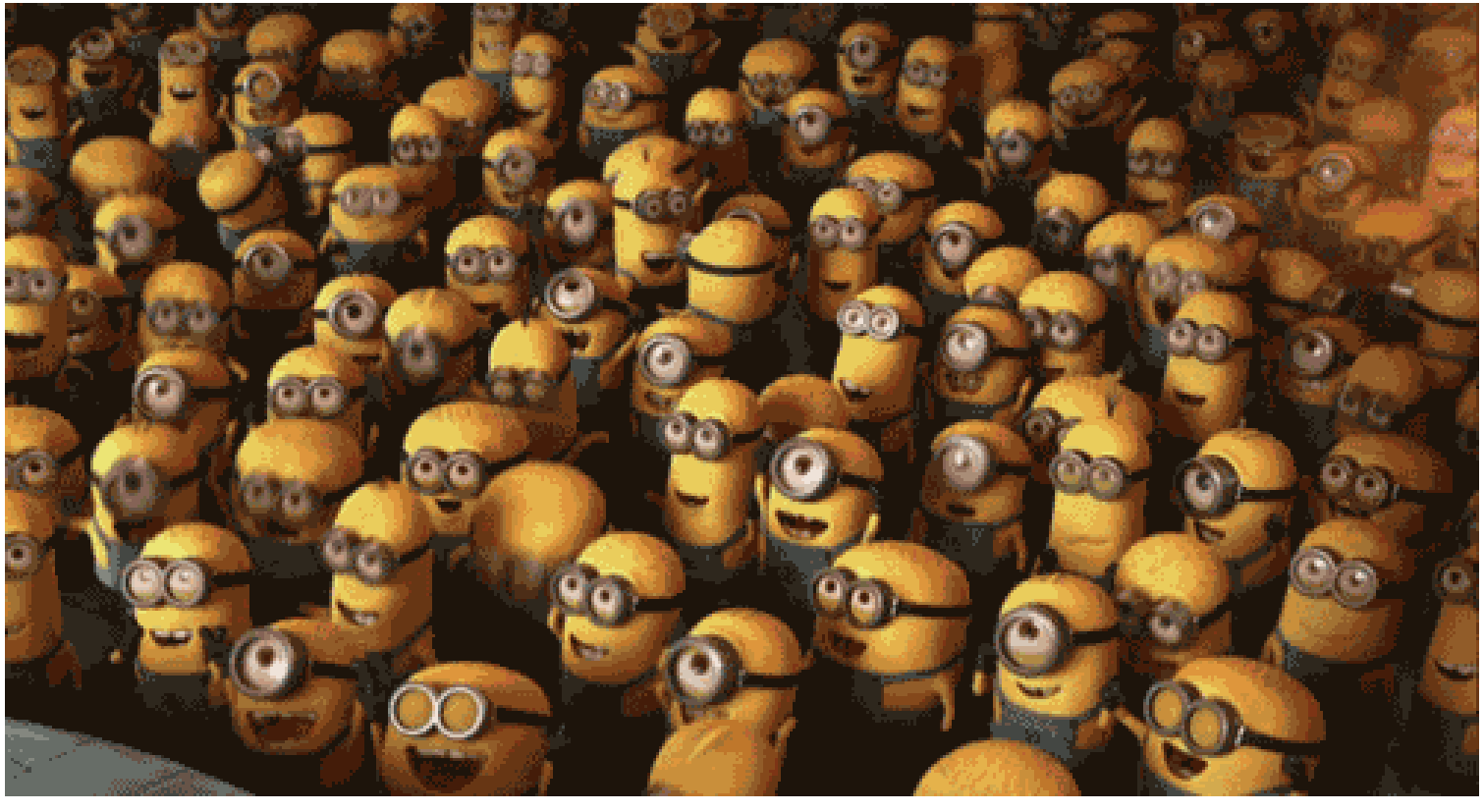
References

- Aaron Bertrand: https://blogs.sentryone.com/aaronbertrand/backtobasics-ctes/
- Essential SQL: https://www.essentialsql.com/recursive-ctes-explained/
- Microsoft: https://docs.microsoft.com/en-us/sql/t-sql/queries/with-common-table-expression-transact-sql?view=sql-server-2014
- Red Gate: https://www.red-gate.com/simple-talk/sql/t-sql-programming/sql-server-cte-basics/

Copyright © 2026, SQLRV, LLC - All Rights Reserved

Thanks to our Sponsors!

Without them, we wouldn’t be here!!



Copyright © 2026, SQLRV LLC - All Rights Reserved

Questions & Comments

BONUS:
A **TON** of free eBooks from RedGate and others!

PRESENTATION FEEDBACK:
- Your thoughts needed
- Improve presentations
- Make this event even more valuable!!!

Aaron N. Cutshall

www.linkedin.com/in/sqlrv
aaron@sqlrv.com
www.sqlrv.com
www.youtube.com/@sqlrv

Copyright © 2026, SQLRV LLC - All Rights Reserved